Automatic Hardware Design Tool Based on Reusing Transformation

Chongzhou Fang, Zaichen Zhang, Xiaohu You and Chuan Zhang

November 1, 2019

[1] Lab of Efficient Architectures for Digital-communication and Signal-processing (LEADS)
[2] National Mobile Communications Research Laboratory, Southeast University, Nanjing, China
[3] Purple Mountain Laboratories, Nanjing, China

Outline

Introduction

Software

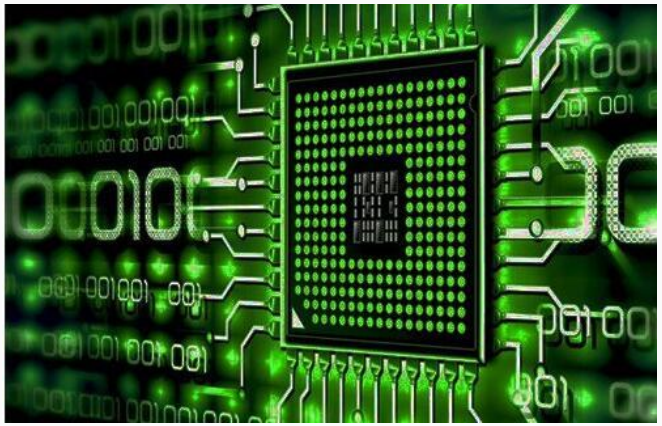
Results

Conclusion

Introduction

Overview

- Complexity
- Tools:
  - ●.1 High level synthesis tools
  - ●.2 Hardware generators[1]



[1] P. Milder, F. Franchetti, J. C. Hoe, and M. P¨uschel, "Computer generation of hardware for linear digital signal processing transforms," ACM Transactions on Design Automation of Electronic Systems, vol. 17, no. 2, p. 15, 2012.

Our Contribution

1. A data flow representation method.

Our Contribution

1. A data flow representation method.
2. A universal algorithm that performs reusing transformation.

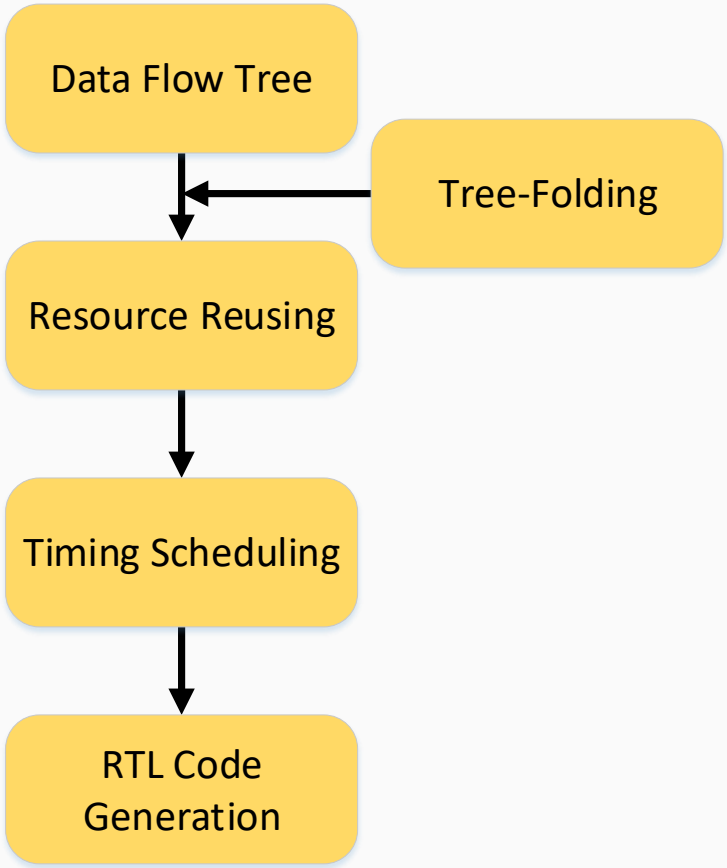
Our Contribution

1. A data flow representation method.
2. A universal algorithm that performs reusing transformation.
3. An automatic hardware generator.

Software

Software

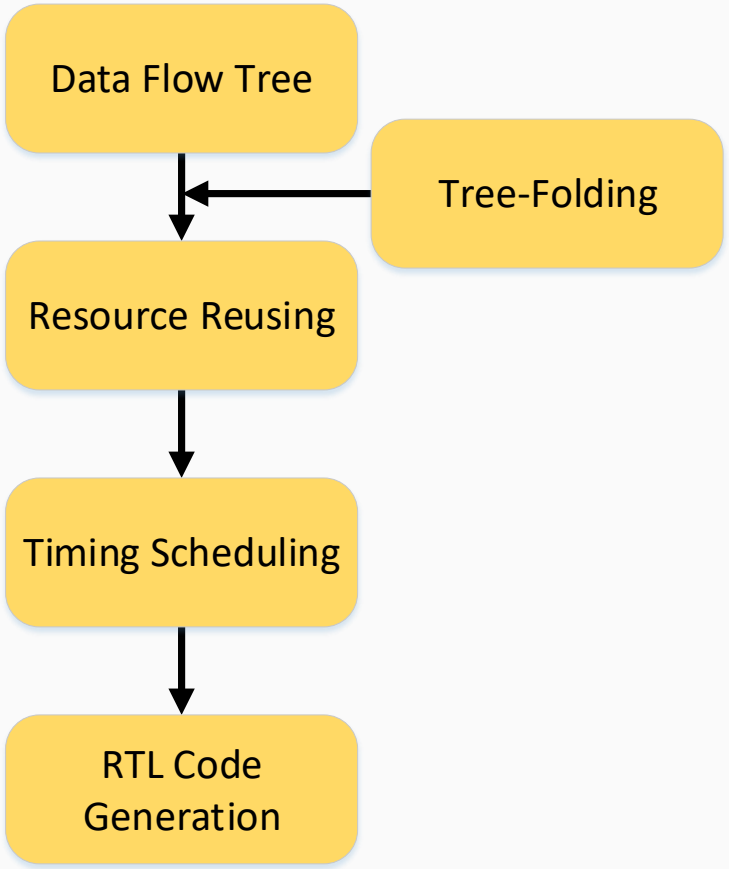
1. Data flow representation



Software

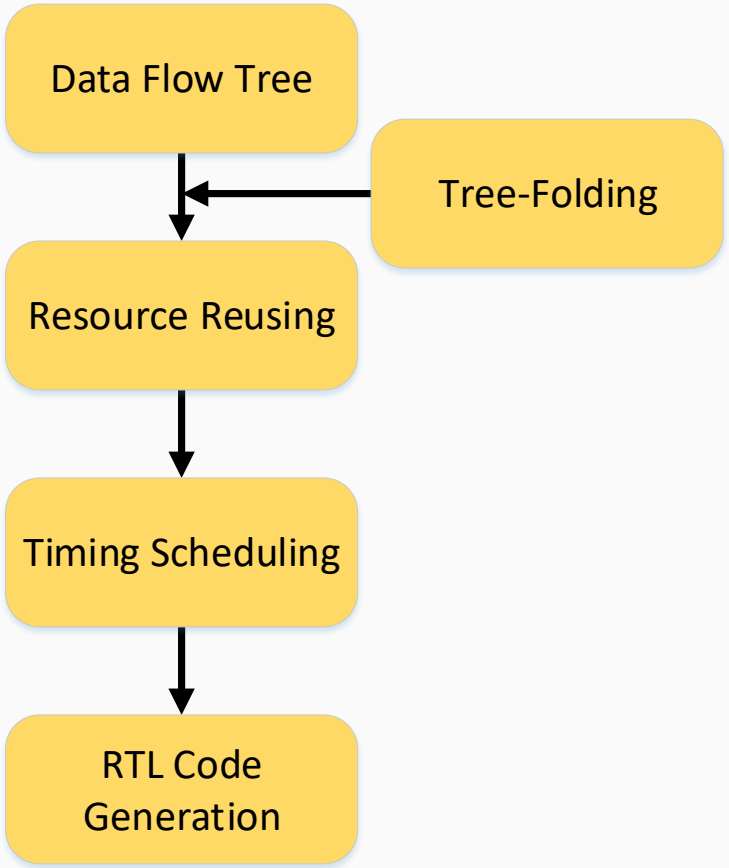
1. Data flow representation

2. Basic operation: tree-folding



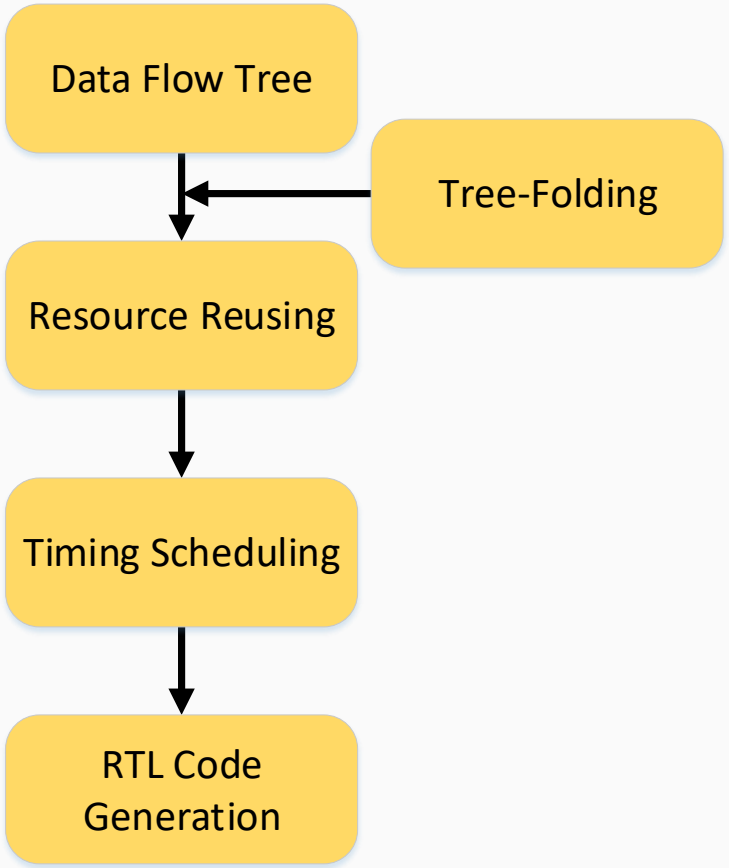
Software

1. Data flow representation
2. Basic operation: tree-folding
3. Resource reusing



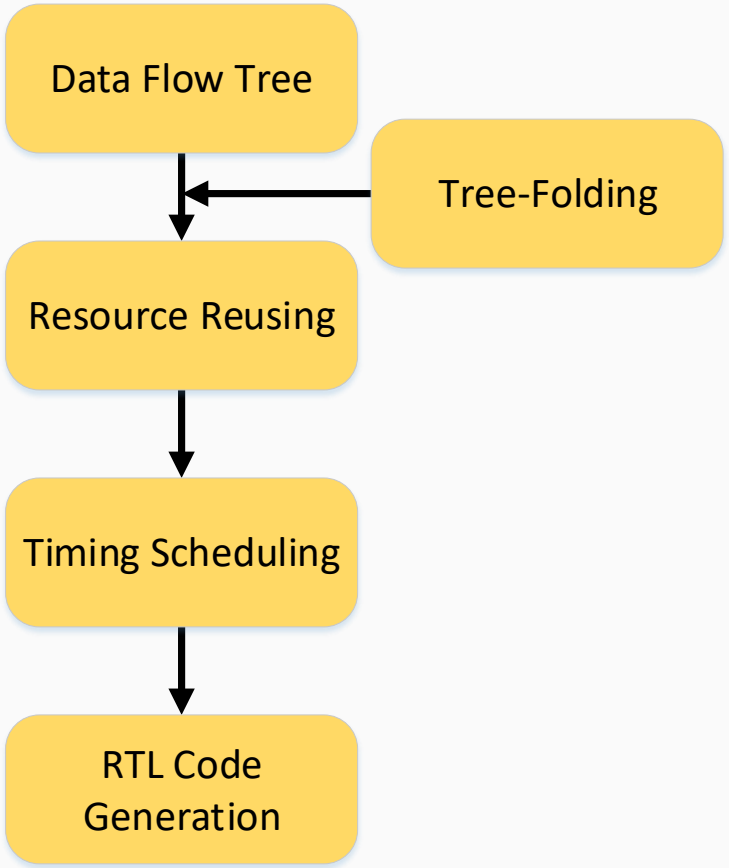
Software

1. Data flow representation
2. Basic operation: tree-folding
3. Resource reusing
4. Timing scheduling



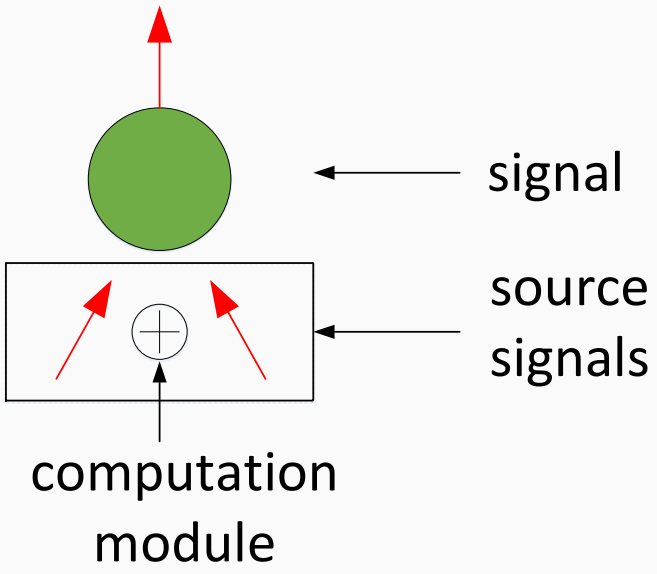
Software

1. Data flow representation
2. Basic operation: tree-folding
3. Resource reusing
4. Timing scheduling
5. RTL code generation



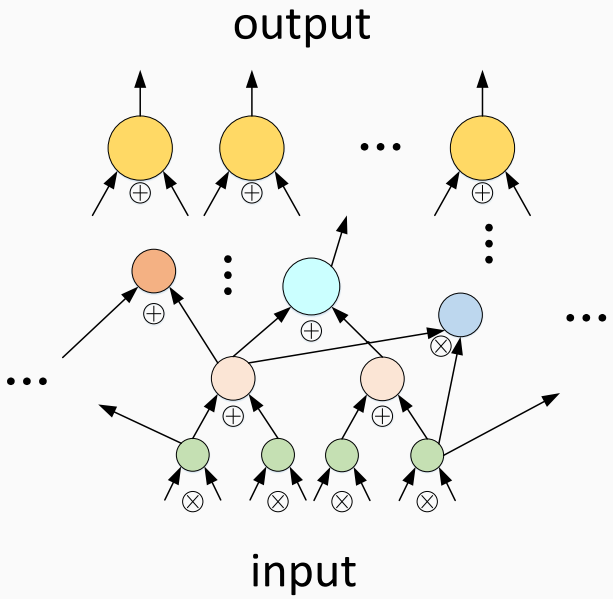
Data Flow Representation

Tree node:



Data Flow Representation

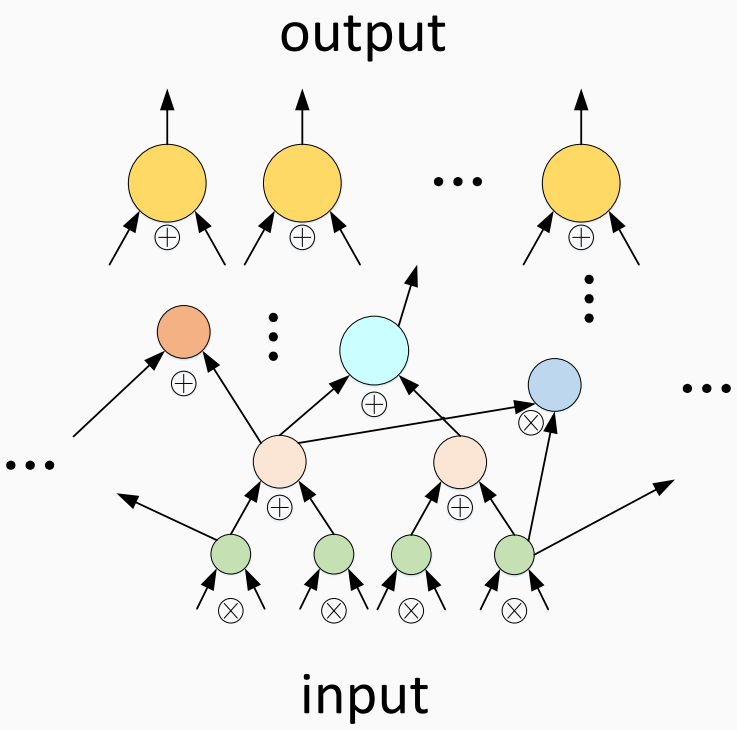
Tree structure:



Data Flow Representation

Advantages:

1. Explicitly provide data dependencies.

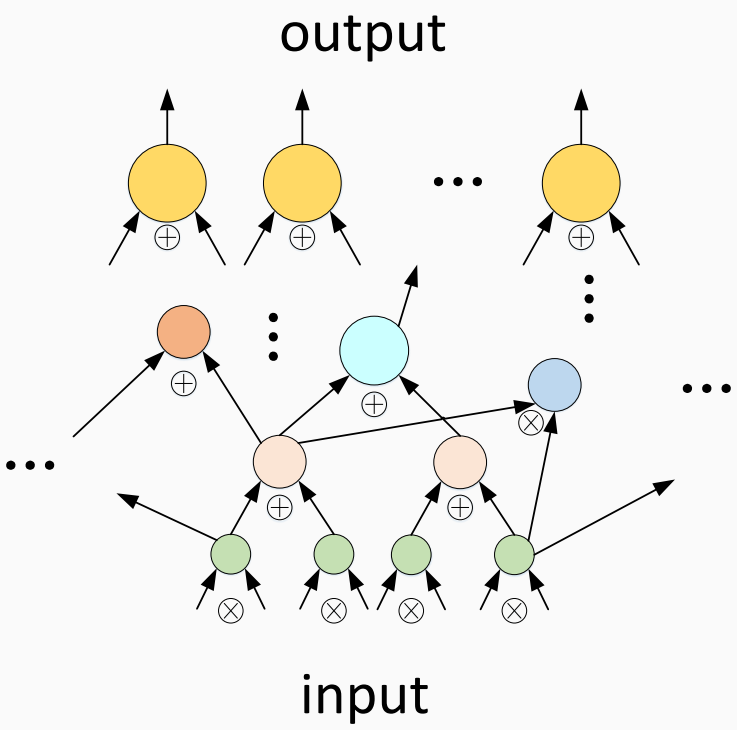


Data Flow Representation

Advantages:

1. Explicitly provide data dependencies.

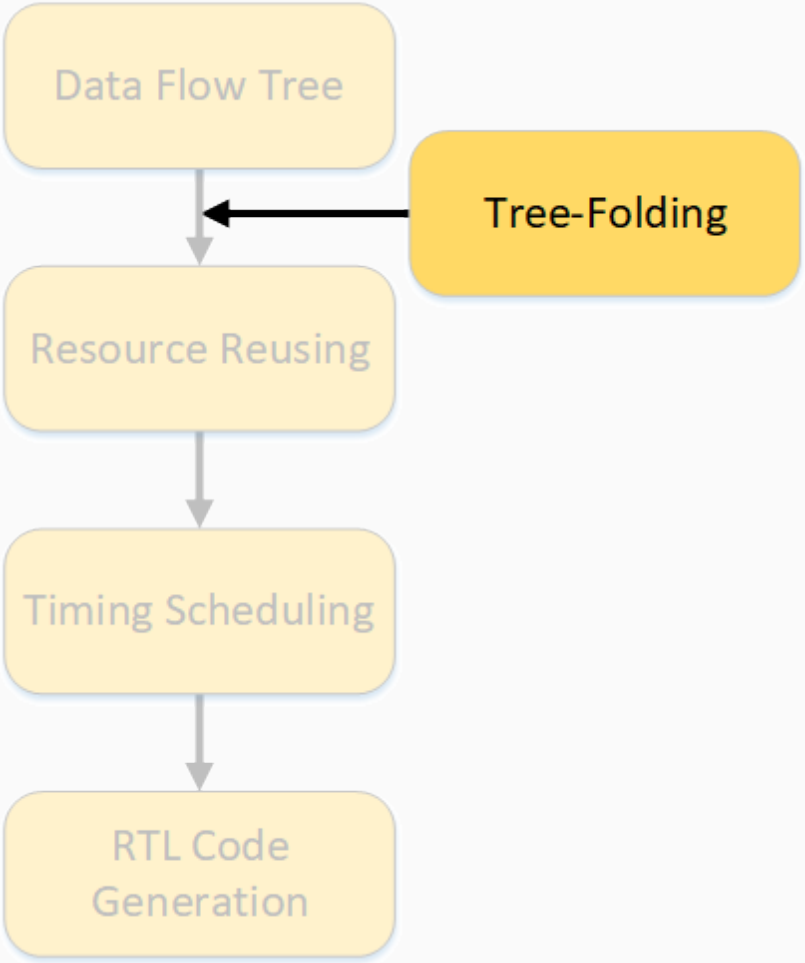
2. Avoid unnecessary operations on lower level nodes.



Tree-Folding Operation

Our algorithm:

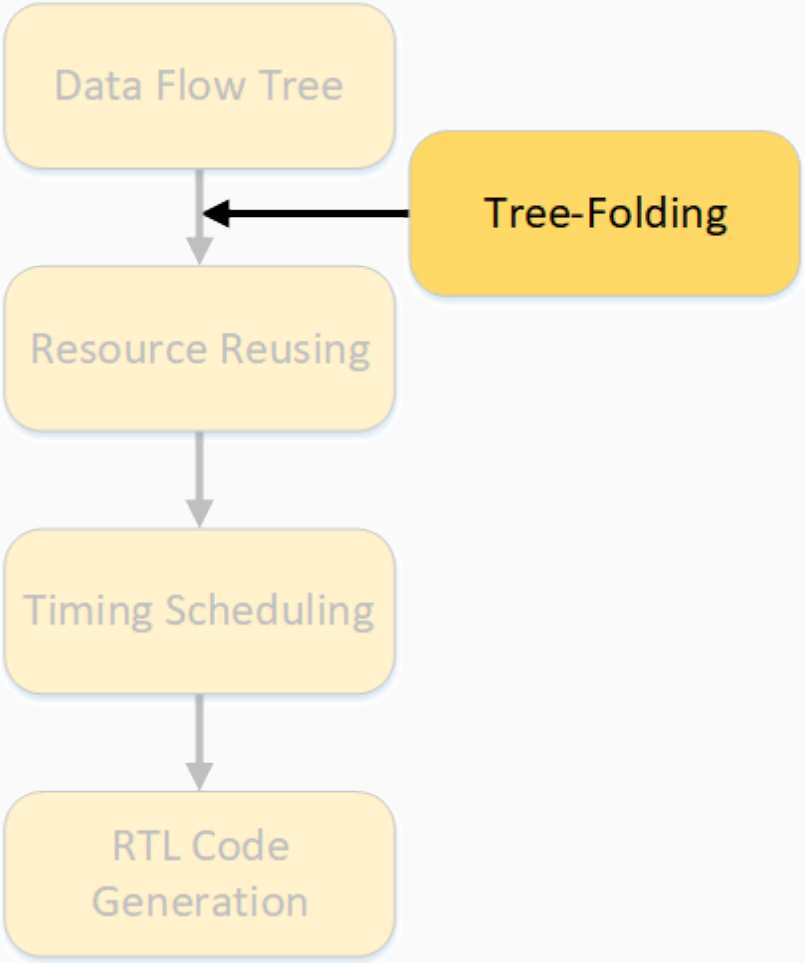
1. Simple replacement of nodes.



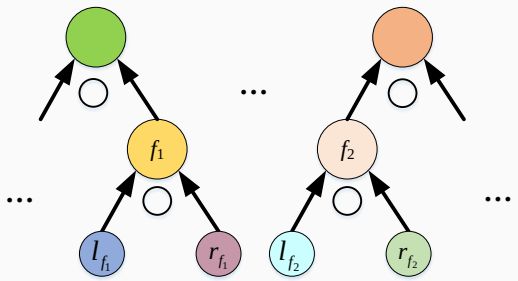
Tree-Folding Operation

Our algorithm:

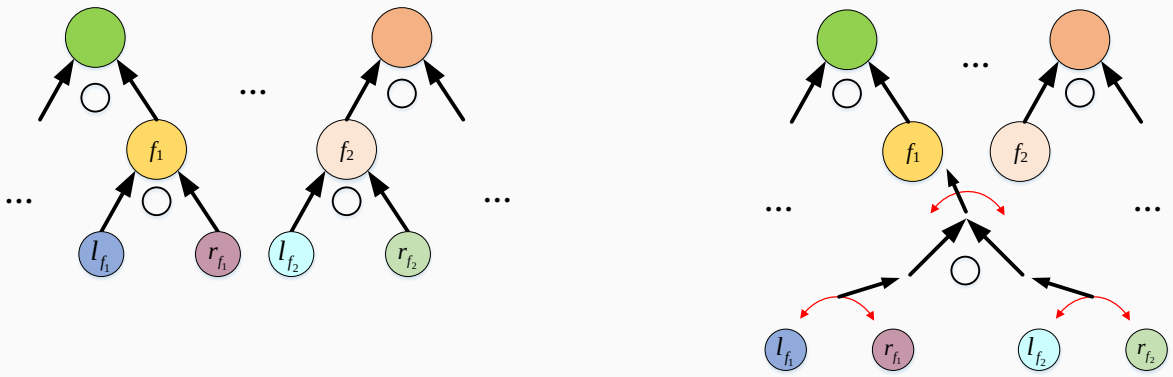
1. Simple replacement of nodes.
2. Renew the whole tree each time.



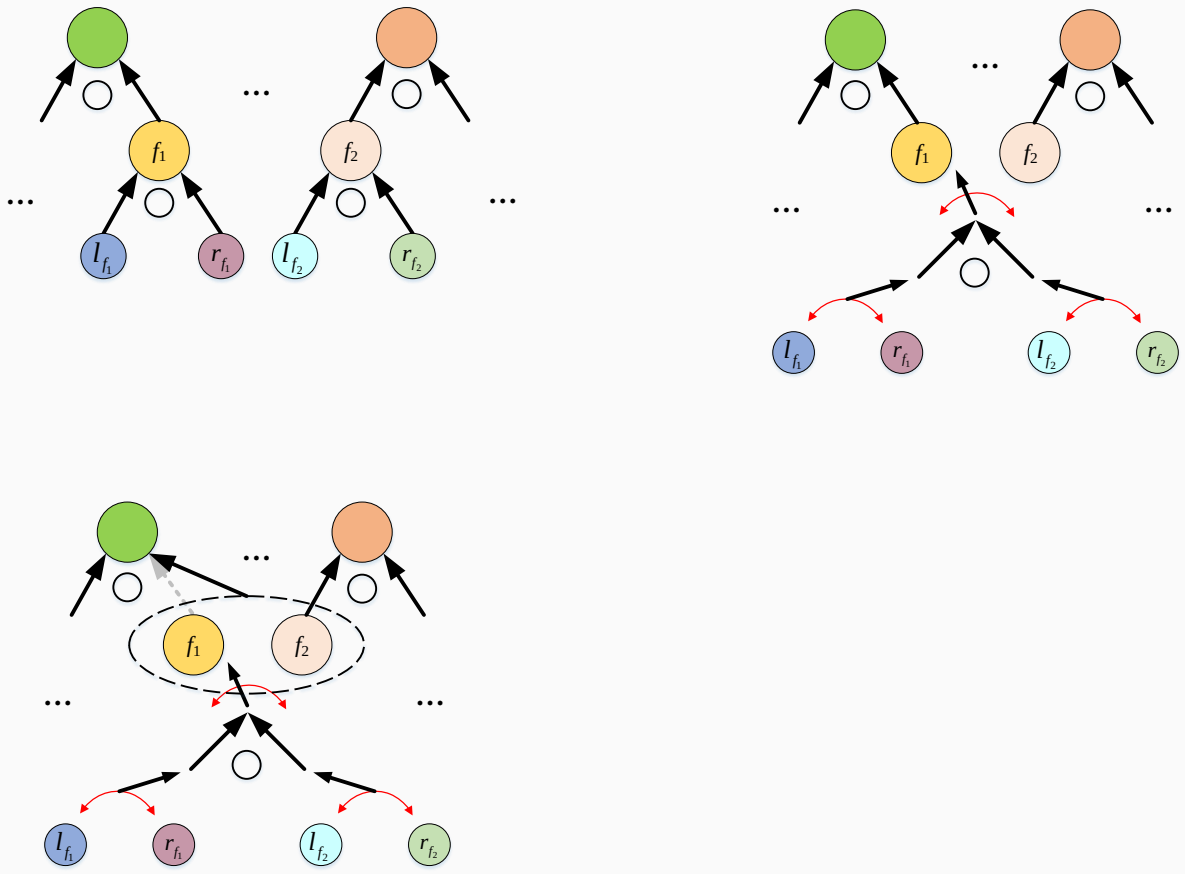
Tree-Folding Operation



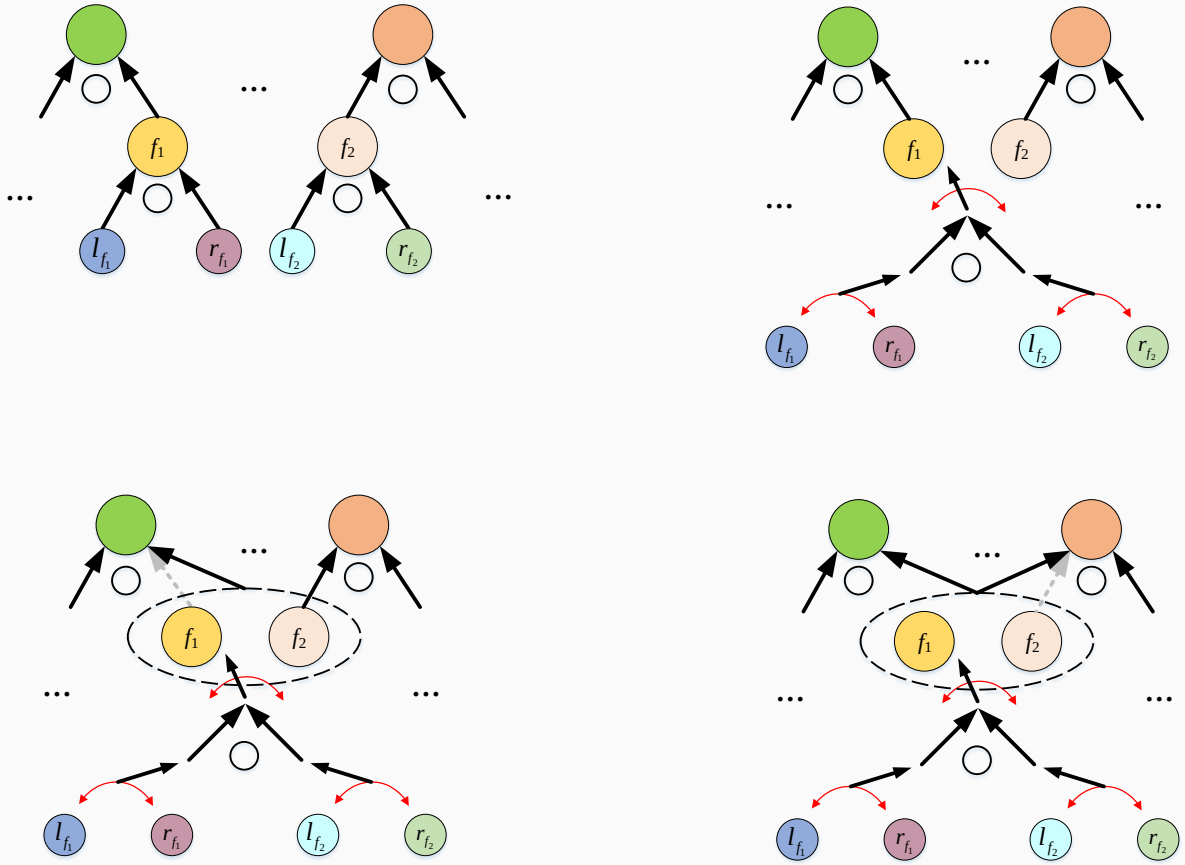
Tree-Folding Operation



Tree-Folding Operation



Tree-Folding Operation



Tree-Folding Operation

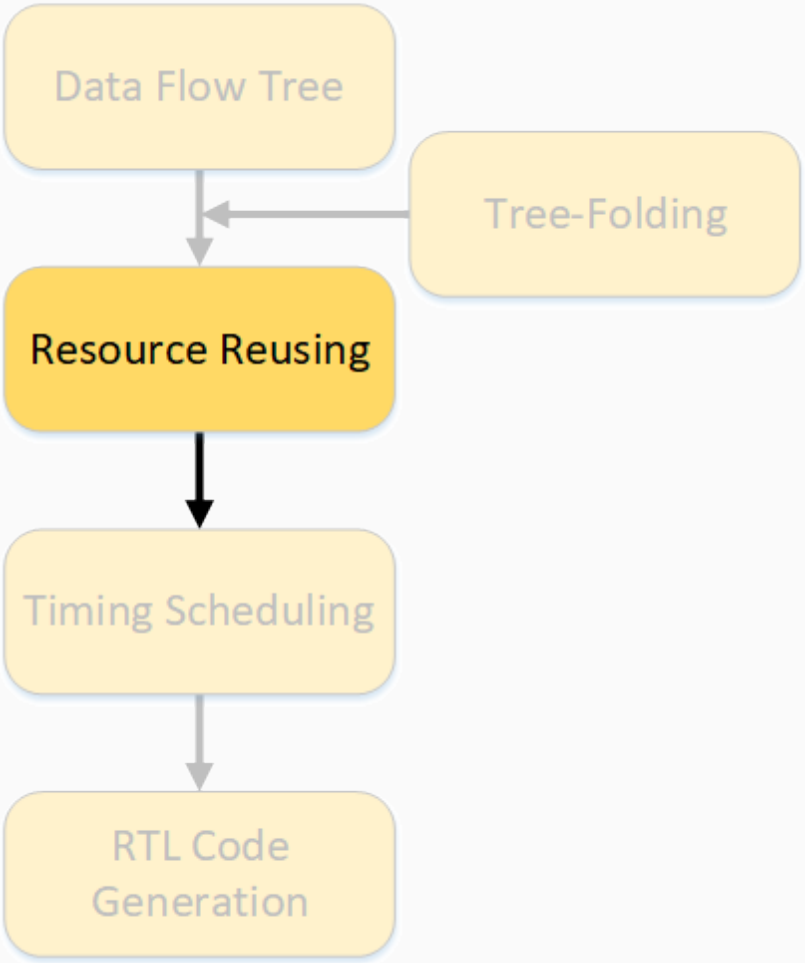
Algorithm 1 Tree-folding operation.

Require: A set of tree nodes $S = \{n_p : (m_{n_p}, l_{n_p}, r_{n_p})\}$ and node identifiers f_1, f_2 corresponding to nodes with the same computation module m_f that are to be reused.

Add $f_1 \& f_2 : (m_f, \{l_{f_1} \& l_{f_2}\}, \{r_{f_1} \& r_{f_2}\})$ to S.
Delete node f_1 and f_2 from S.
for k in S **do**
    if f_1 in l_k or f_1 in r_k **then**
        Replace f_1 in the child node l_k or r_k with $\{f_1 \& f_2 | f_1\}$.
    end if
    if f_2 in l_k or f_2 in r_k **then**
        Replace f_2 in the child node l_k or r_k with $\{f_1 \& f_2 | f_2\}$.
    end if
end for
Adjust the sequence of signals in each node according to their signal identifier.
return S.

Resource Reusing Strategy

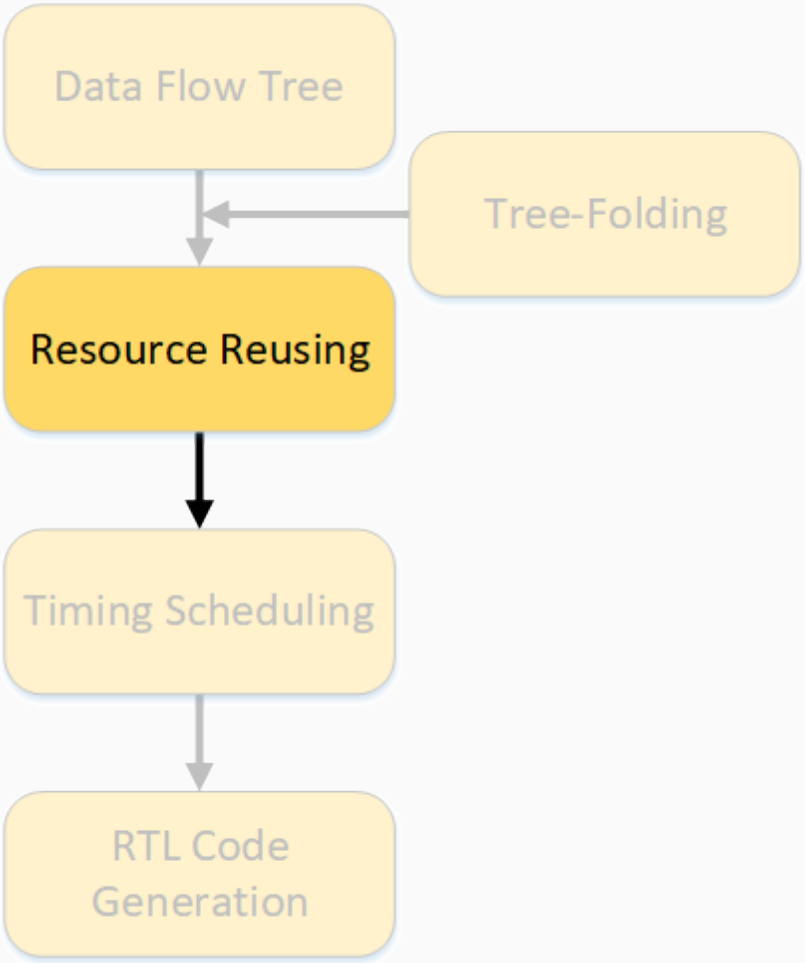
1. Input: Number of required computation modules and a data flow tree.



Resource Reusing Strategy

1. Input: Number of required computation modules and a data flow tree.

2. Method: Recursively using tree-folding operation to compress the tree.

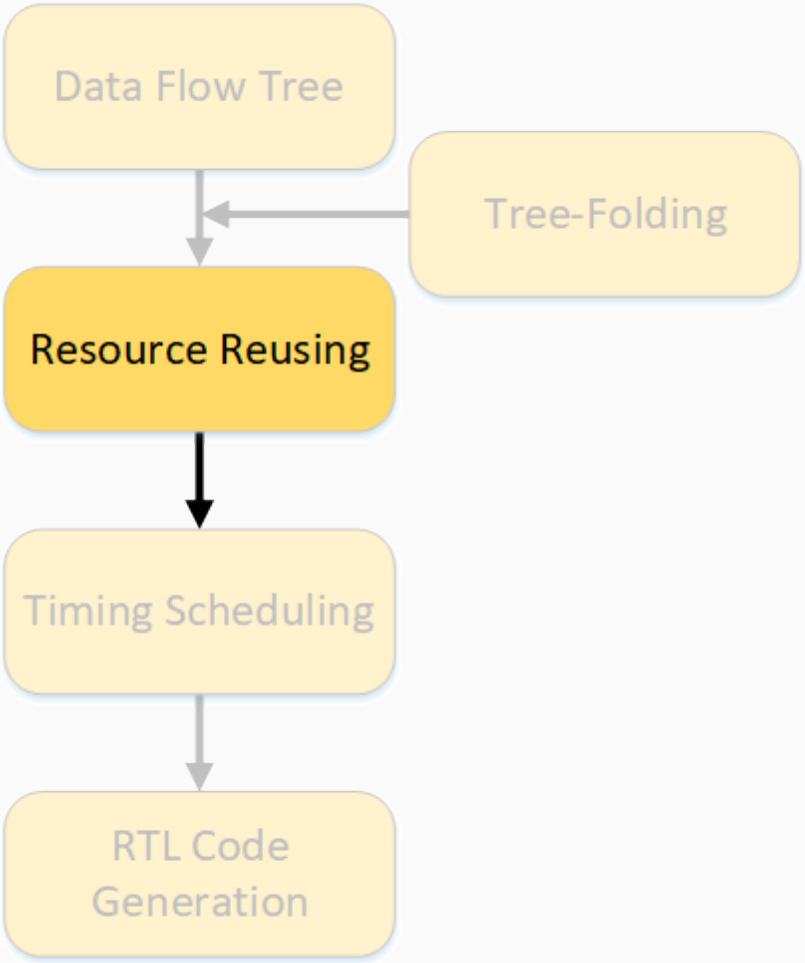


Resource Reusing Strategy

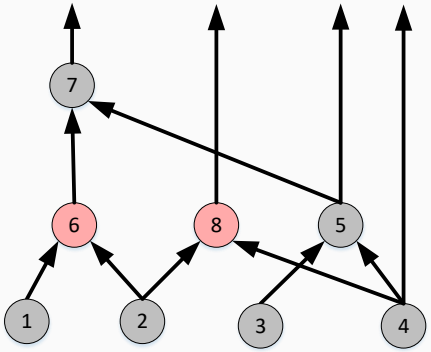
1. Input: Number of required computation modules and a data flow tree.

2. Method: Recursively using tree-folding operation to compress the tree.

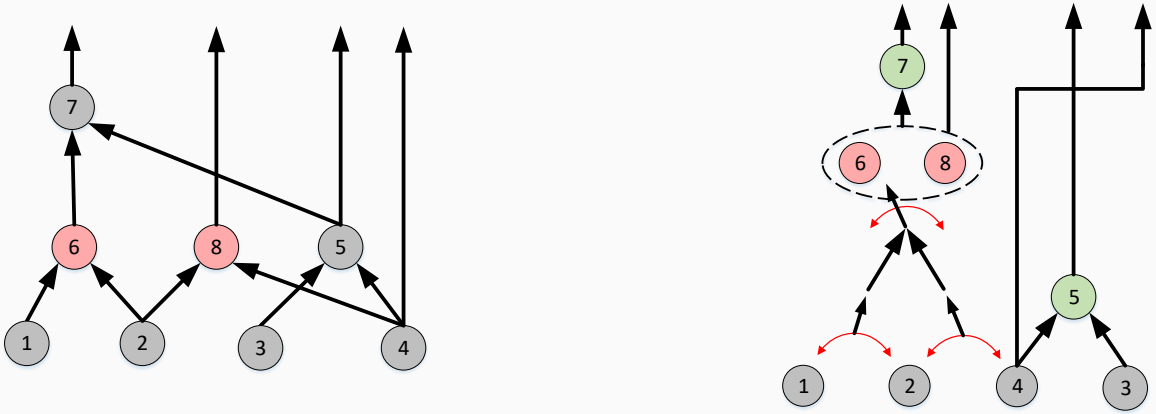
3. Output: A compressed data flow tree that satisfies resource constraints.



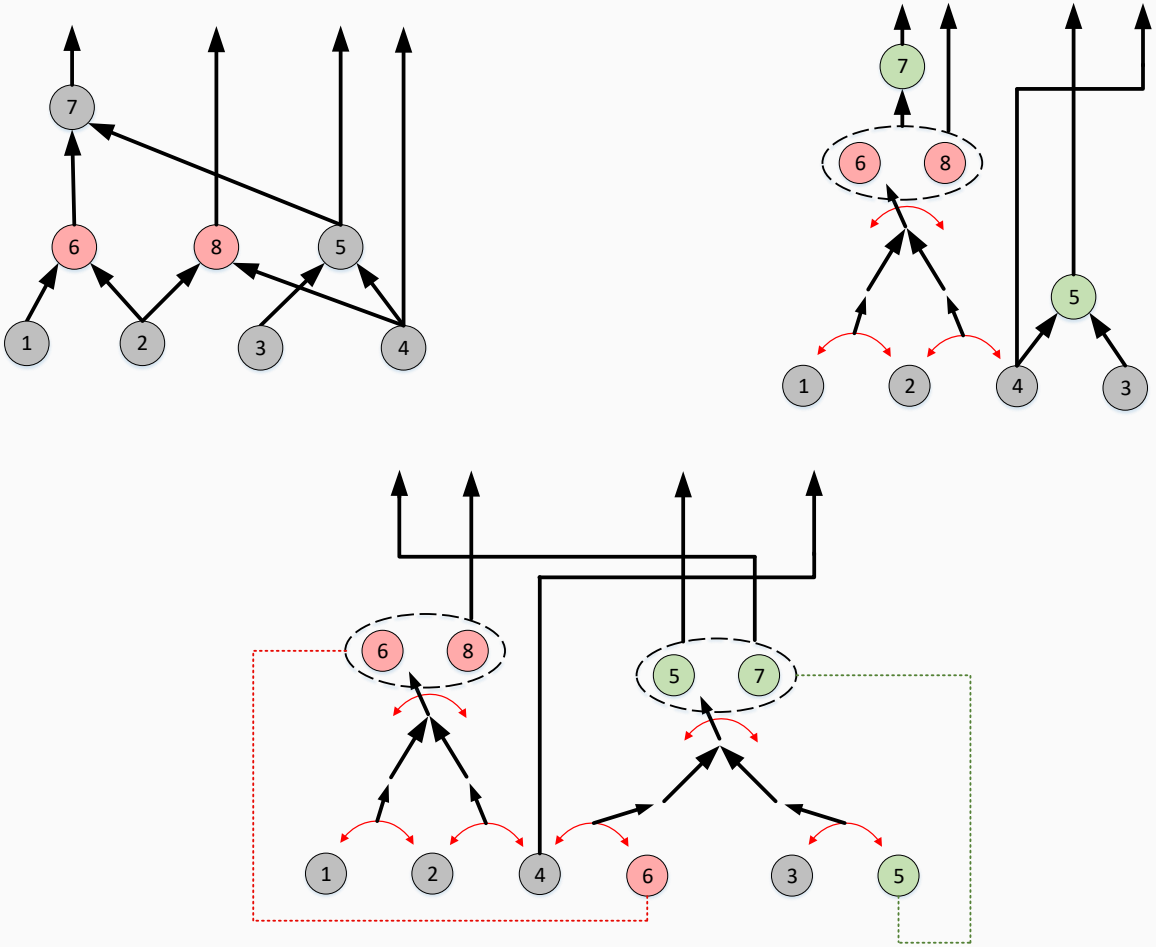
Resource Reusing Strategy



Resource Reusing Strategy



Resource Reusing Strategy



Resource Reusing Strategy

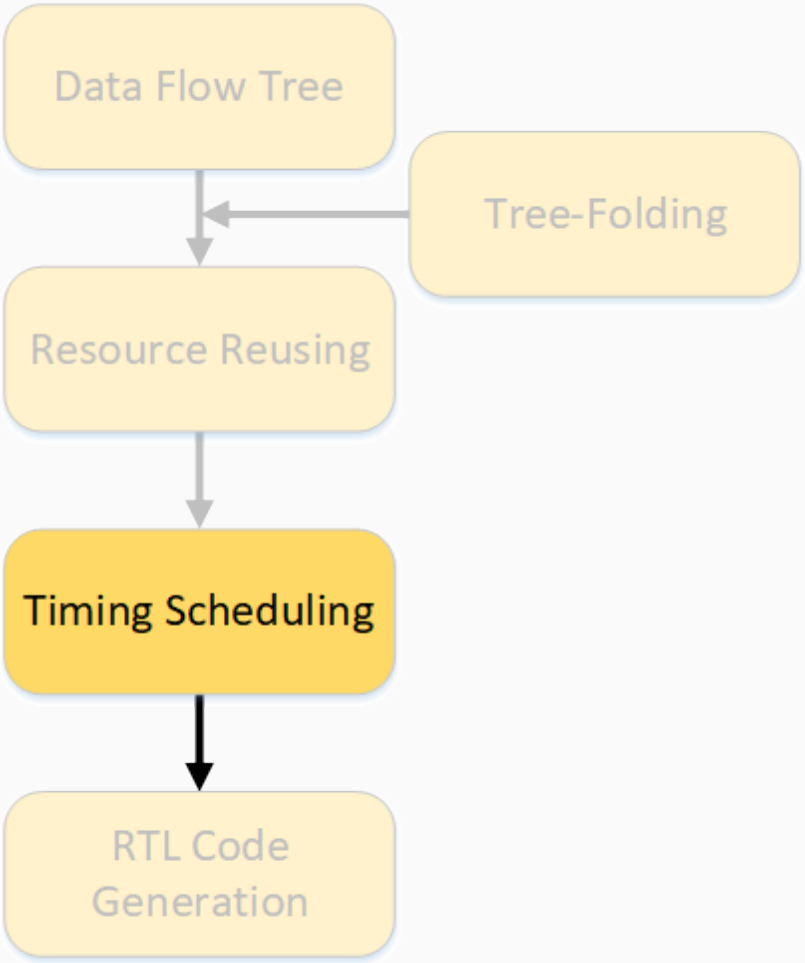
Algorithm 2 Resource reusing strategy.

Require: A set of unfolded tree nodes
$S = \{n_p : (m_{n_p}, l_{n_p}, r_{n_p})\}$, number of available computation modules n_0, computation module type m_0.

```
Find Δn which is the discrepancy between number of tree nodes and n_0.
Let R = 1, ℒ = { } where R is the number of processing cycles required in current state and ℒ is the set of nodes available for tree-folding.
while Δn > 0 do
    for k in S do
        if less than R signals are folded in node k then
            Add k to ℒ.
        end if
    end for
    for x in ℒ and |ℒ| ≥ 2 do
        y = the succeeding element of x in ℒ.
        S = Tree-folding(S, x, y).
        Delete x,y in ℒ.
        Δn = Δn − 1.
        if Δn = 0 then
            return S.
        end if
    end for
    R = R + 1, ℒ = { }.
end while
return S.
```

Timing Scheduling

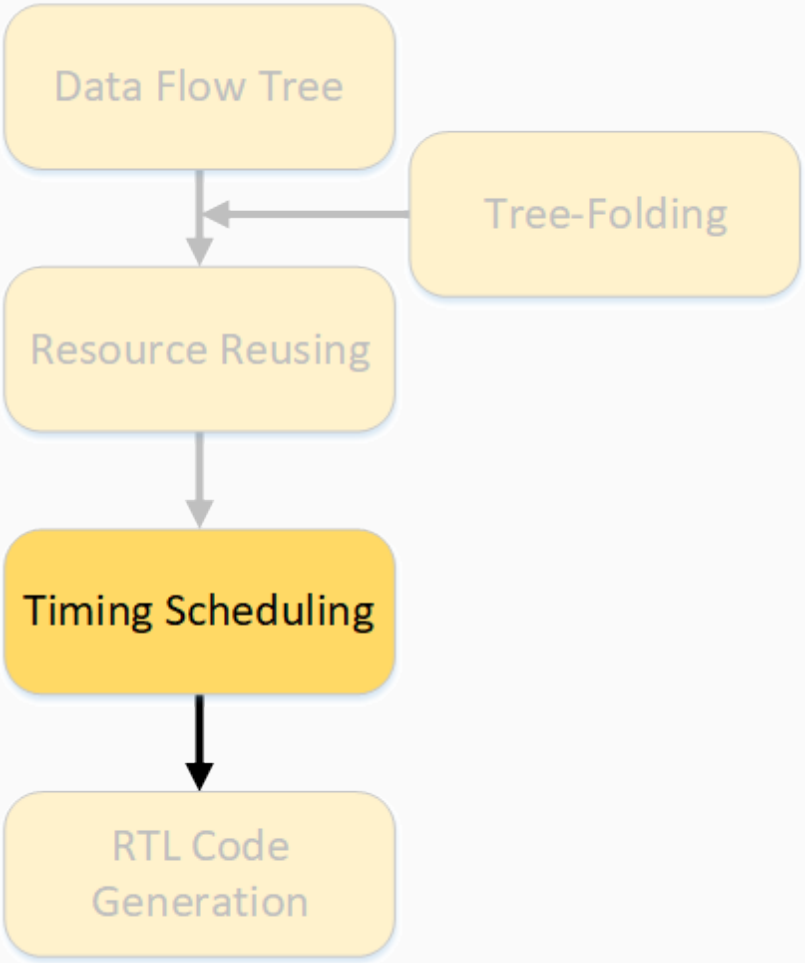
1. Final step before code generation.



Timing Scheduling

1. Final step before code generation.

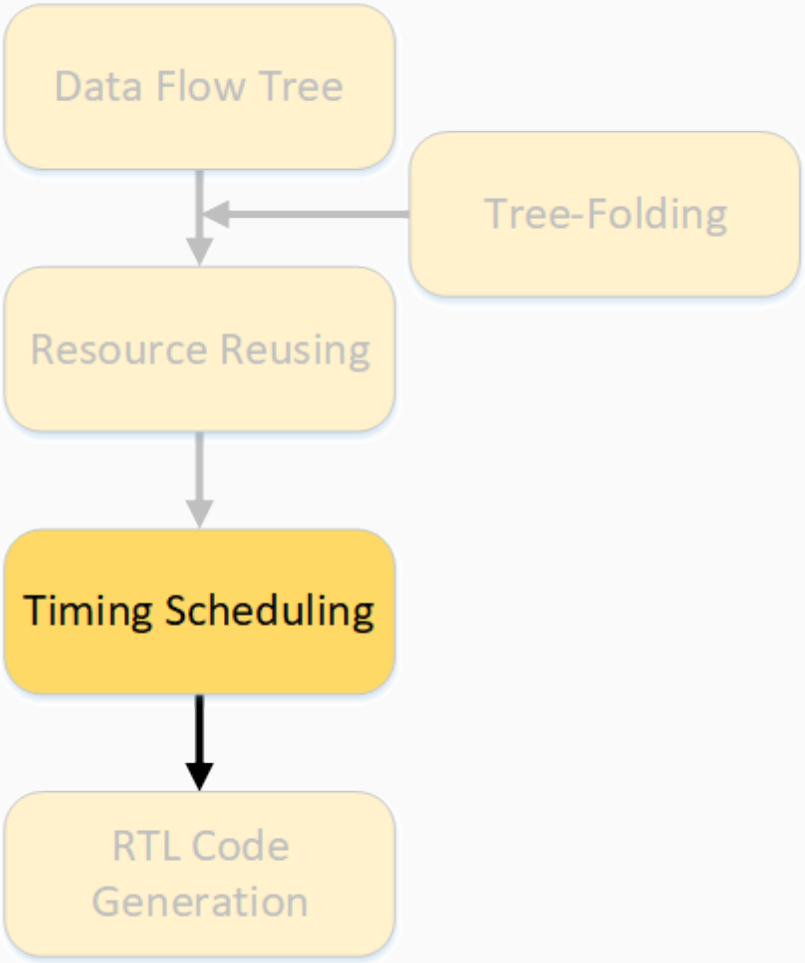
2. Construct a time table for all signals.



Timing Scheduling

Steps:

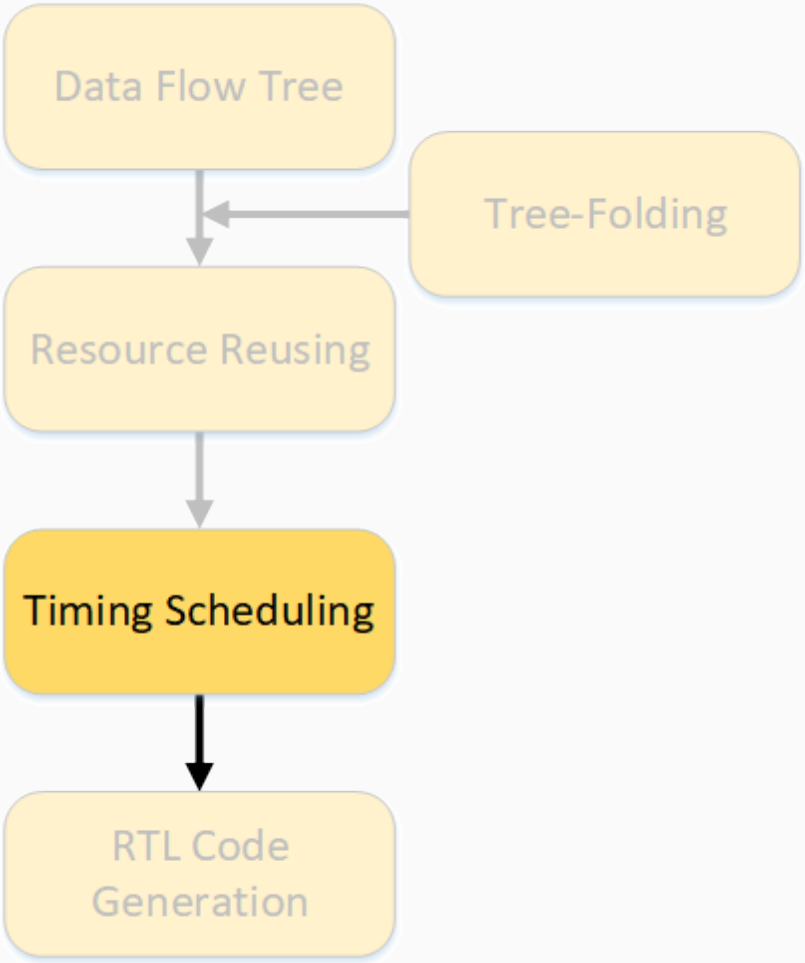
1. Obtain an initial time table according to node information.



Timing Scheduling

Steps:

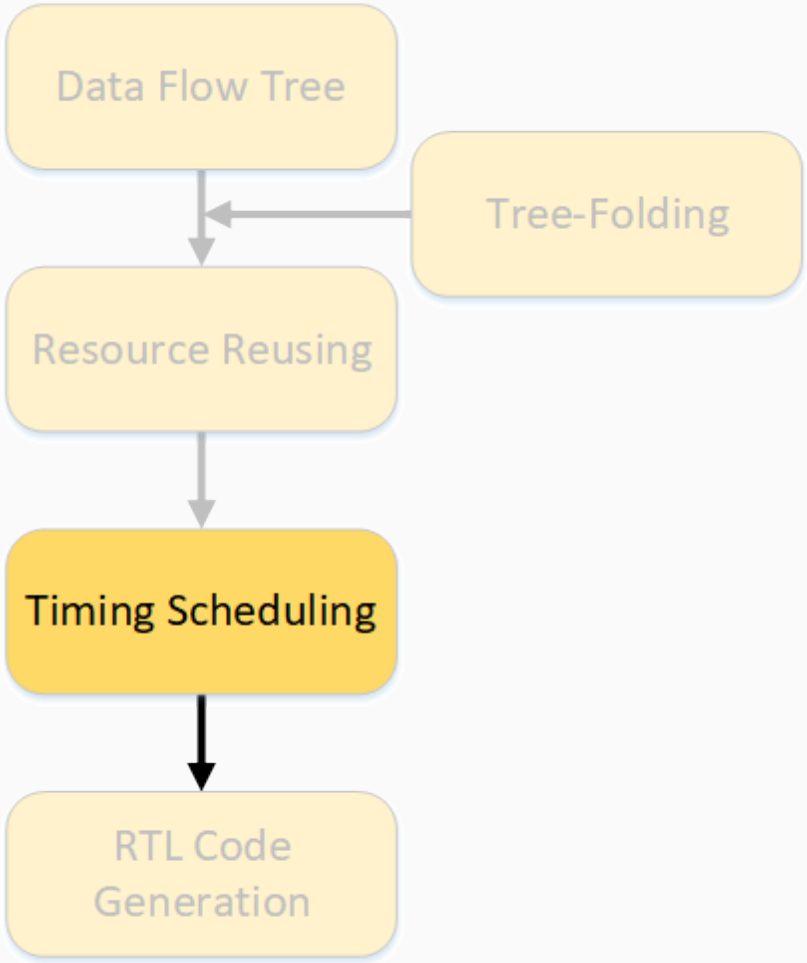
1. Obtain an initial time table according to node information.
2. Check for conflicts:



Timing Scheduling

Steps:

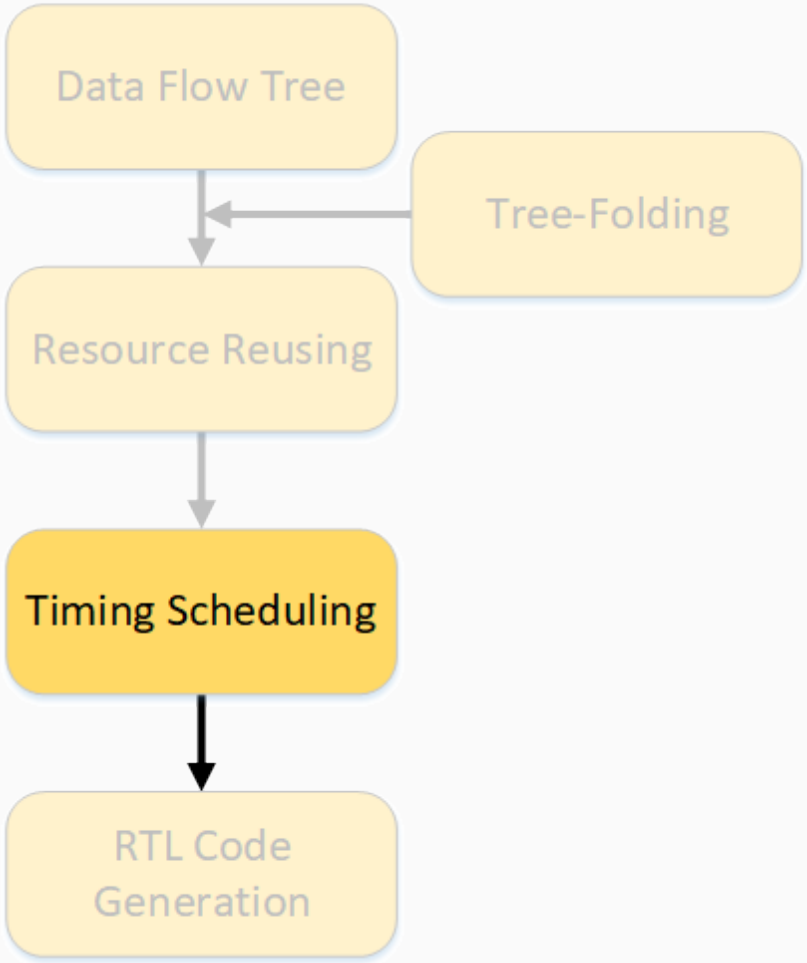
1. Obtain an initial time table according to node information.
2. Check for conflicts:
   - parent signals are generated earlier than child signals



Timing Scheduling

Steps:

1. Obtain an initial time table according to node information.
2. Check for conflicts:
   - parent signals are generated earlier than child signals
   - signals that are folded together are generated at the same time

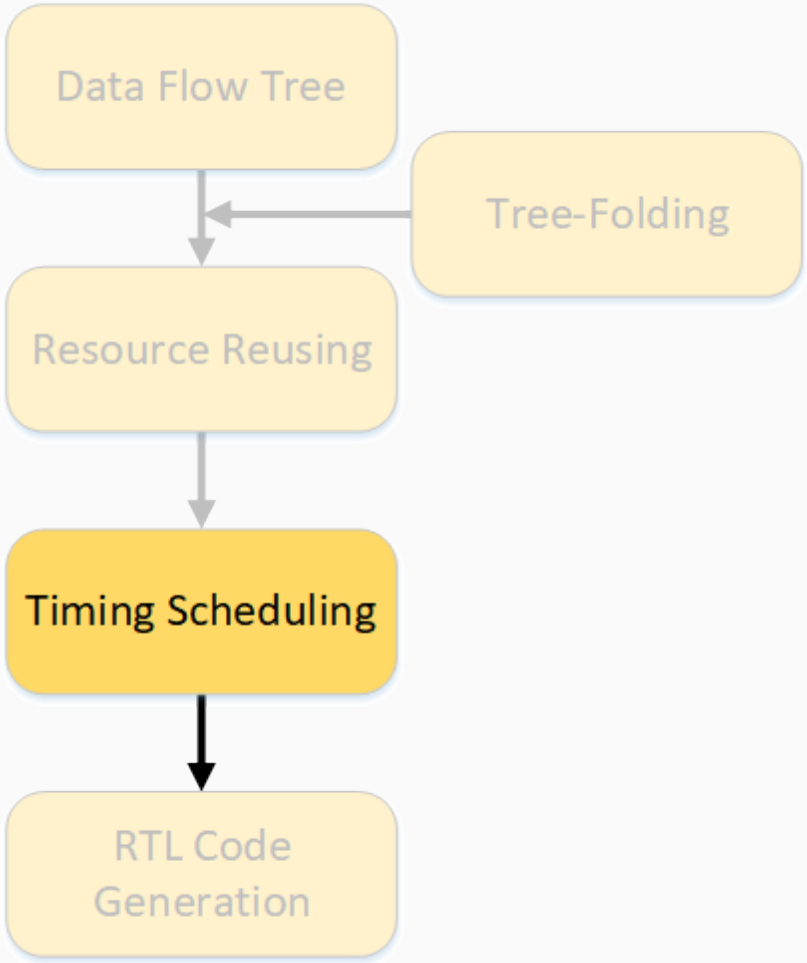


Timing Scheduling

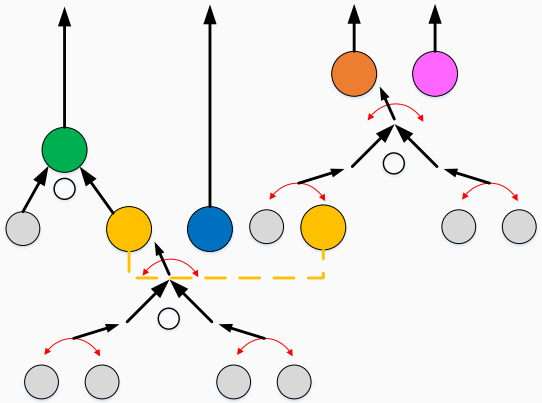
Steps:

1. Obtain an initial time table according to node information.
2. Check for conflicts:
   - parent signals are generated earlier than child signals
   - signals that are folded together are generated at the same time

   then postpone corresponding signals.

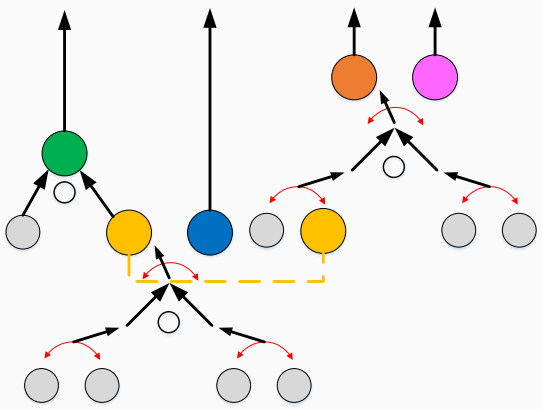


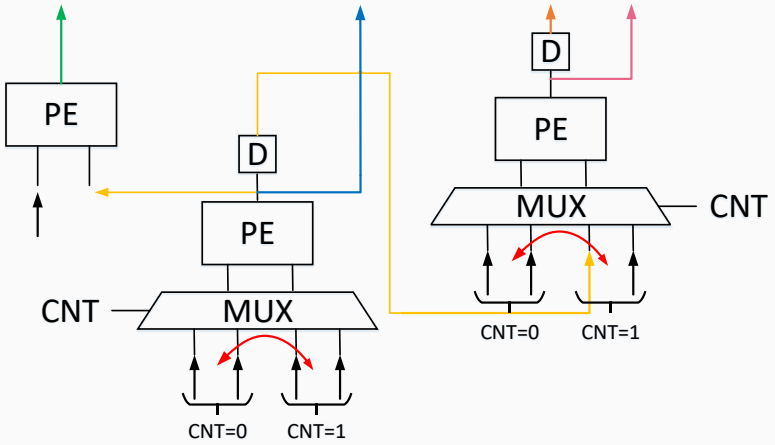
Circuit Template



1. Employ MUXs controlled by a global counting signal.
2. Insert registers.

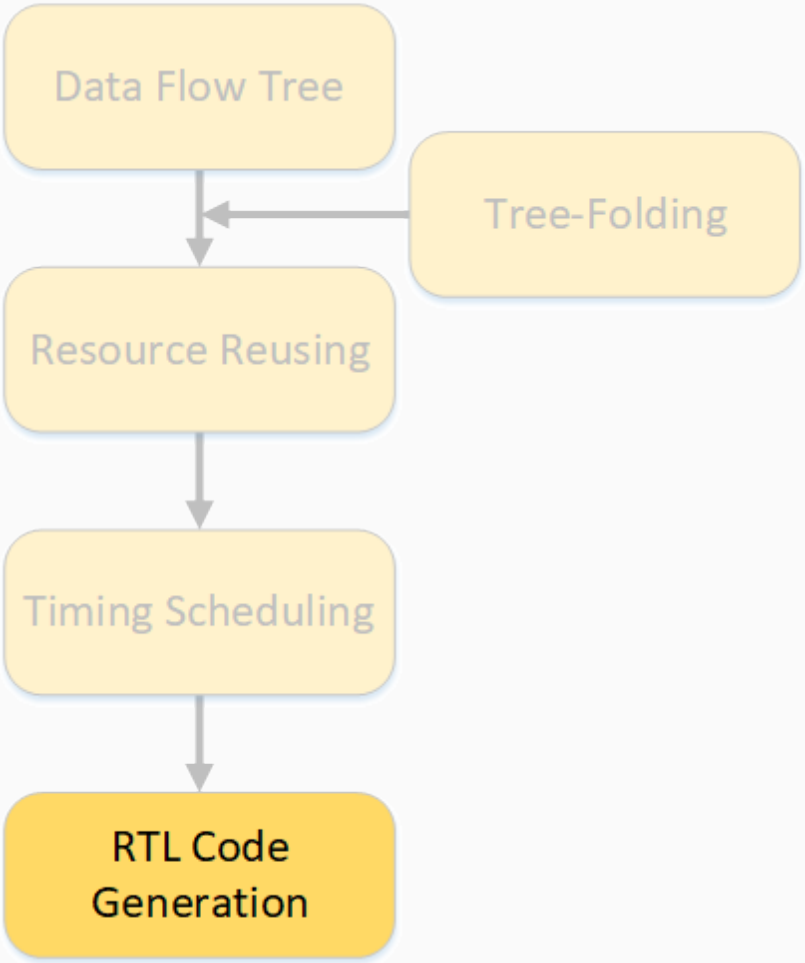
Circuit Template





Code Generation

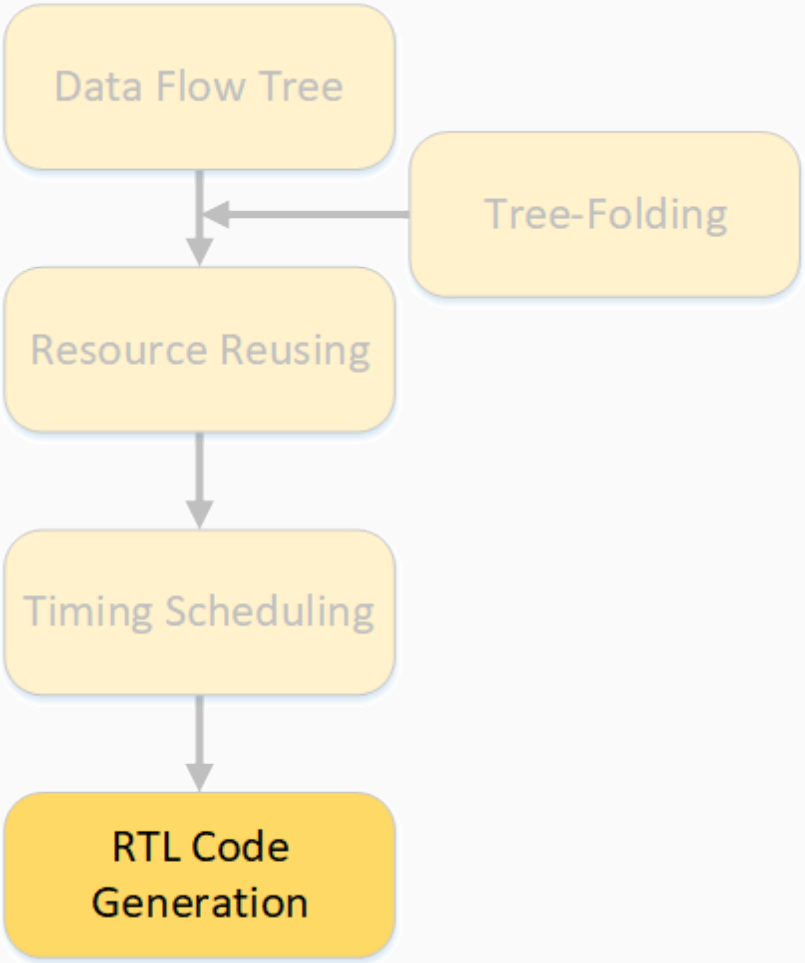
- Generate synthesizable Verilog RTL code.



Code Generation

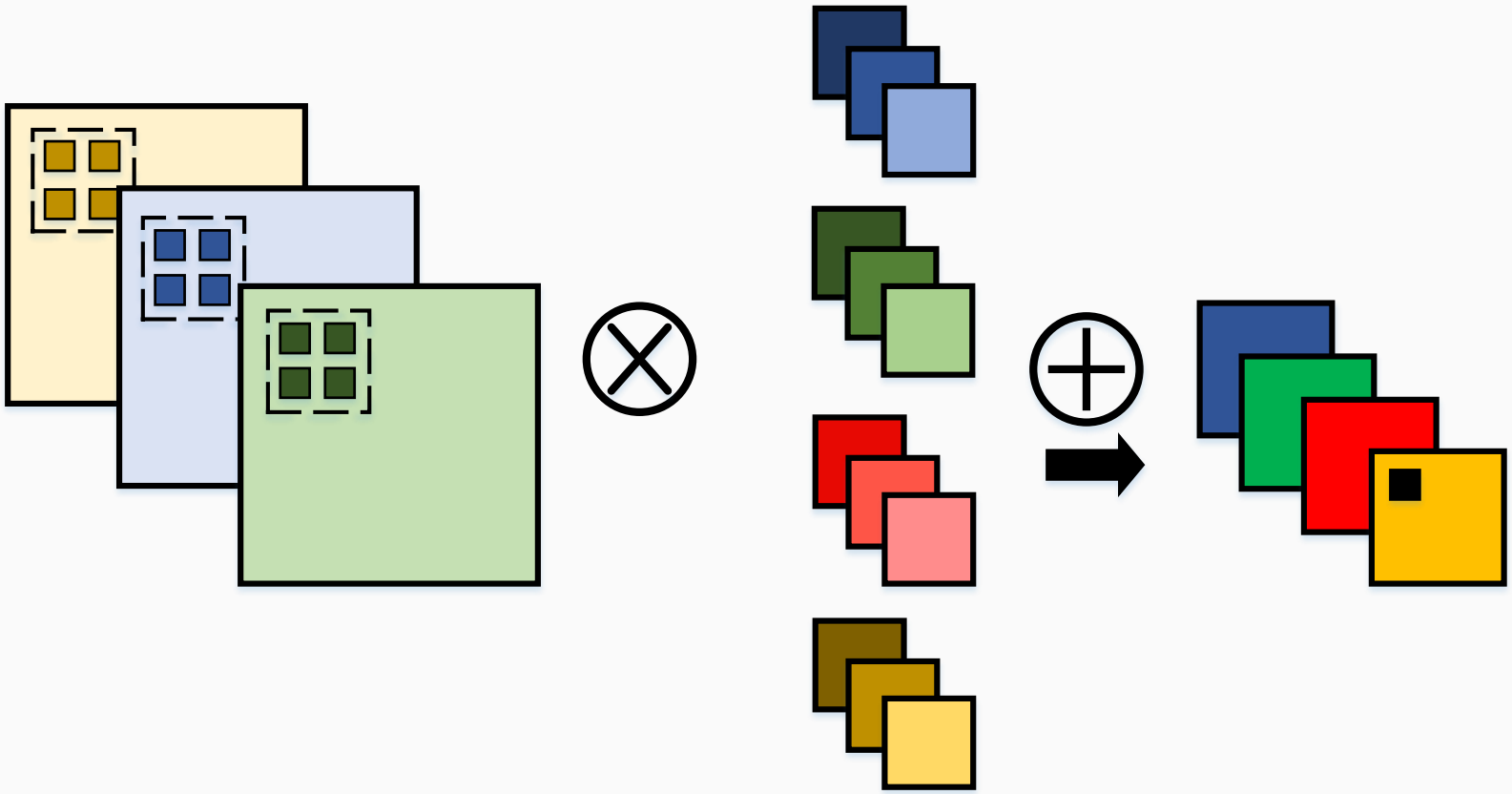
- Generate synthesizable Verilog RTL code.

- Map data flow tree to circuit with the help of the time table and circuit template.



Results

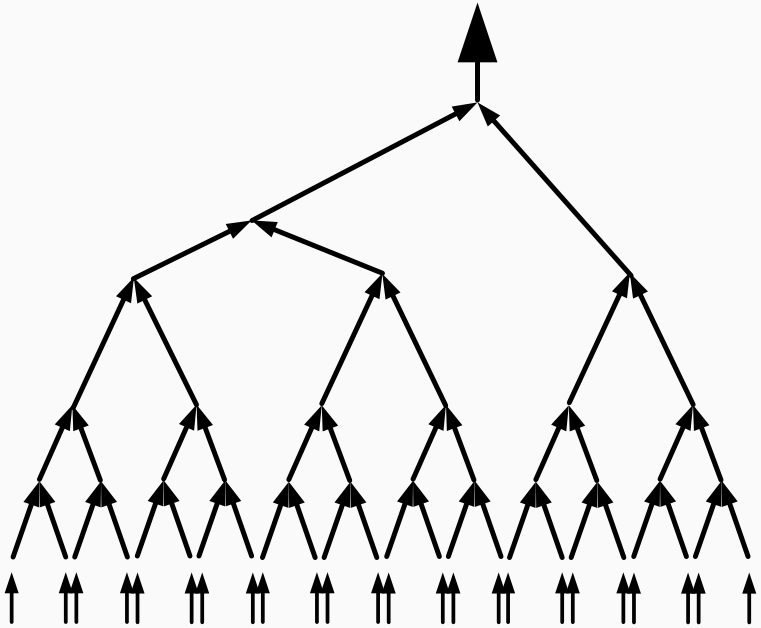
Experiment Settings



Experiment Settings

1. Test under different user specifications.

2. Provide FPGA synthesis results.



Results

Table 1: Results under different reusing configurations.

(n_m, n_a)	LUT	LUTRAM	DSP	FF	CCs
$(12, 11)$	176	0	12	0	1
$(9, 5)$	274	0	9	437	6
$(6, 6)$	337	0	6	227	4
$(6, 2)$	316	16	6	248	8
$(4, 4)$	294	0	4	358	6
$(4, 2)$	260	0	4	230	8
$(2, 2)$	260	0	2	135	8
$(1, 1)$	229	0	1	134	14

Conclusion

Conclusion

1. We propose a set of algorithms that perform resource reusing.
2. We develop hardware generator which automatically generates synthesizable Verilog RTL code.
3. We test on a simple example and validate our theory.

Q & A